```
import os
import re
import string
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import jieba
import torch
from torch import nn
from tqdm.notebook import tqdm
from transformers import BertTokenizer, BertModel, BertForMaskedLM
```

```
def remove_special_symbols(text, symbols=[]):
    try:
        if not isinstance(text, str):
            text = str(text)
        text = text.strip()
        for symbol in symbols:
            text = re.sub(symbol, "", text)
    except:
        pass
    return text
```

```
shanghai_path = 'data/shanghai_listings-2020.10.24.csv'
df = pd.read_csv(shanghai_path, encoding='utf-8')
```

```
/tmp/ipykernel_2898/1965088461.py:2: DtypeWarning: Columns (8,22,30,34,37,48,53,57,67) have mixed
types. Specify dtype option on import or set low_memory=False.
  df = pd.read_csv(shanghai_path, encoding='utf-8')
```

```
for column in df.columns:
    print(f'{column}: {df.loc[:, column].isnull().sum()}')
```

```
id: 0
listing_url: 0
scrape_id: 0
last_scraped: 0
name: 2
description: 2299
neighborhood_overview: 10453
picture_url: 0
host_id: 0
host_url: 0
host_name: 9
host_since: 4
host_location: 109
host_about: 13405
host_response_time: 3775
host_response_rate: 3774
host_acceptance_rate: 4982
host_is_superhost: 4
host_thumbnail_url: 4
host_picture_url: 6
host_neighbourhood: 13501
host_listings_count: 4
host_total_listings_count: 4
host_verifications: 0
host_has_profile_pic: 4
host_identity_verified: 4
neighbourhood: 10453
neighbourhood_cleansed: 4
neighbourhood_group_cleansed: 35568
latitude: 0
longitude: 0
property_type: 0
room_type: 0
accommodates: 4
bathrooms: 35568
bathrooms_text: 60
bedrooms: 631
beds: 385
```

```
amenities: 0
price: 0
minimum_nights: 0
maximum_nights: 0
minimum_minimum_nights: 0
maximum_minimum_nights: 0
minimum_maximum_nights: 0
maximum_maximum_nights: 0
minimum_nights_avg_ntm: 0
maximum_nights_avg_ntm: 4
calendar_updated: 35568
has_availability: 0
availability_30: 0
availability_60: 0
availability_90: 0
availability_365: 0
calendar_last_scraped: 0
number_of_reviews: 0
number_of_reviews_ltm: 0
number_of_reviews_l30d: 2
first_review: 16601
last_review: 16601
review_scores_rating: 16906
review_scores_accuracy: 16934
review_scores_cleanliness: 16933
review_scores_checkin: 16937
review_scores_communication: 16935
review_scores_location: 16937
review_scores_value: 16939
license: 35568
instant_bookable: 0
calculated_host_listings_count: 0
calculated_host_listings_count_entire_homes: 0
calculated_host_listings_count_private_rooms: 0
calculated_host_listings_count_shared_rooms: 2
reviews_per_month: 16603
```

# 1. Data preparetion

```
# Read data
beijing_path = 'data/beijing_listings-2020.10.24.csv'
df = pd.read_csv(beijing_path, encoding='utf-8')
```

## 1.1. Show the first 5 rows

```
df.head(5)
```

| | id | listing_url | scrape_id | last_scraped | name | description | neighborhood |
|---|---|---|---|---|---|---|---|
| **0** | 114384 | https://www.airbnb.com/rooms/114384 | 2.020000e+13 | 2020/10/26 | CBD Luxury 1-bedroom suite with a 30m2 terrace | Located in the core position of Chaoyang CBD. ... | NaN |
| **1** | 114465 | https://www.airbnb.com/rooms/114465 | 2.020000e+13 | 2020/10/26 | 国贸CBD中国尊梵悦108附近豪华总统套房睡3人/步行五分钟至地铁站万达广场沃尔玛 | Located in the core position of Chaoyang CBD. ... | Walking distan dozens of bus Guoma... |
| **2** | 128496 | https://www.airbnb.com/rooms/128496 | 2.020000e+13 | 2020/10/26 | Heart of Beijing: House with View 2 | real life in a real Hutong. Not the Disneyland... | NaN |
| **3** | 161902 | https://www.airbnb.com/rooms/161902 | 2.020000e+13 | 2020/10/26 | cozy studio in center of Beijing | 北京元嘉公寓座落于东直门立交桥东南，邻近机场快线东直门站，同时靠近地铁13号线、2号线和东直... | NaN |
| **4** | 162144 | https://www.airbnb.com/rooms/162144 | 2.020000e+13 | 2020/10/26 | nice studio near subway, sleep 4 | The space<br />Extended stay self-cater... | NaN |

5 rows × 74 columns

## Check how many null in each column

```
# Check column is null
for column in df.columns:
    print(f'{column}: {df.loc[:, column].isnull().sum()}')
```

```
id: 0
listing_url: 0
scrape_id: 0
last_scraped: 0
name: 1
description: 2539
neighborhood_overview: 9983
picture_url: 0
host_id: 0
host_url: 0
host_name: 12
host_since: 0
host_location: 107
host_about: 11155
host_response_time: 3820
host_response_rate: 3820
host_acceptance_rate: 4863
host_is_superhost: 0
```

```
host_thumbnail_url: 0
host_picture_url: 1
host_neighbourhood: 15549
host_listings_count: 0
host_total_listings_count: 0
host_verifications: 0
host_has_profile_pic: 0
host_identity_verified: 1
neighbourhood: 9983
neighbourhood_cleansed: 1
neighbourhood_group_cleansed: 27438
latitude: 0
longitude: 0
property_type: 0
room_type: 0
accommodates: 1
bathrooms: 27438
bathrooms_text: 54
bedrooms: 399
beds: 259
amenities: 0
price: 0
minimum_nights: 0
maximum_nights: 0
minimum_minimum_nights: 0
maximum_minimum_nights: 0
minimum_maximum_nights: 0
maximum_maximum_nights: 0
minimum_nights_avg_ntm: 0
maximum_nights_avg_ntm: 1
calendar_updated: 27438
has_availability: 0
availability_30: 0
availability_60: 0
availability_90: 0
availability_365: 0
calendar_last_scraped: 0
number_of_reviews: 0
number_of_reviews_ltm: 0
number_of_reviews_l30d: 1
first_review: 12455
last_review: 12455
review_scores_rating: 12769
review_scores_accuracy: 12769
review_scores_cleanliness: 12769
review_scores_checkin: 12771
review_scores_communication: 12769
review_scores_location: 12775
review_scores_value: 12775
license: 27438
instant_bookable: 0
calculated_host_listings_count: 0
calculated_host_listings_count_entire_homes: 0
calculated_host_listings_count_private_rooms: 0
calculated_host_listings_count_shared_rooms: 1
reviews_per_month: 12455
```

## Randomly check length of a description

```
# Random check length of a description
text = df.description[2743]
len(text)
```

```
330
```

## Show the text

There are some html elements, so first, we have to remove these

```
# Show the text
text
```

```
'霄云里8号是一家高端星级酒店公寓,位于京城繁华的燕莎商圈,附近汇聚着燕莎友谊商城等百货商场以及林立的高档写字楼。<br />这里临近北京国际展览中心、农业展览馆,是商务出差办事的便利下榻居所。此外,距离地铁10号线三元桥站也不远,出租约起步价即到。 <br />酒店具有四大品质特征:地处经济发达城市的中心区域;品位高雅,体现文化和艺术氛围;配套设施满足白领精英人士商务休闲双重需求;I-hotel网络时代的聪明酒店。 <br />这里属公寓式酒店格局,性价比高,酒店外的草坪绿化也增添了惬意之感,闹中取静。<br />房间宽敞,厨房设备均采用德国进口优质品牌,提升整体居住舒适度,同时,房间内的10兆高速网络和大堂无线网络覆盖,最适合现代人快节奏的网络生活。'
```

## Remove some symbols and text

html, chines and english special symbols removal

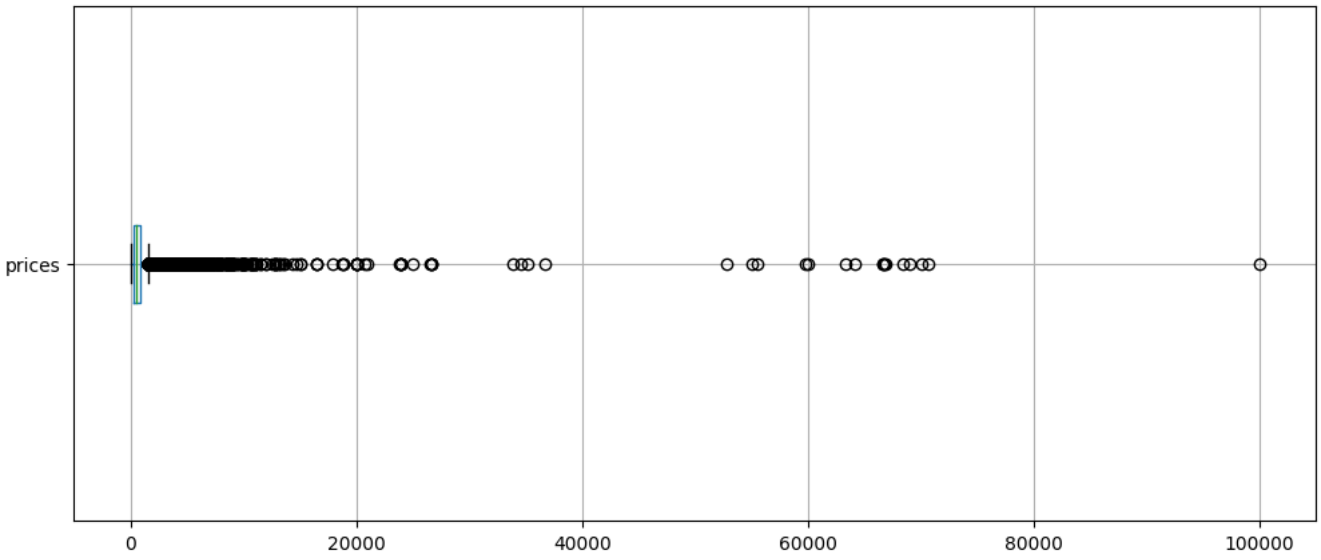
```
# Remove some irrevalant symbol and text
html_symbol = r"<[^>]+>"
chinese_symbol = r"[%s]+" % "，。？！：；【】『』「」“”‘’！·～…、｜■★—"
english_symbol = r"[%s]+" % string.punctuation
# number_symbol = r"\d+"
space = r"\s+"

symbols = [html_symbol, chinese_symbol, english_symbol]
text_filtered = remove_special_symbols(text, symbols)
text_filtered
```

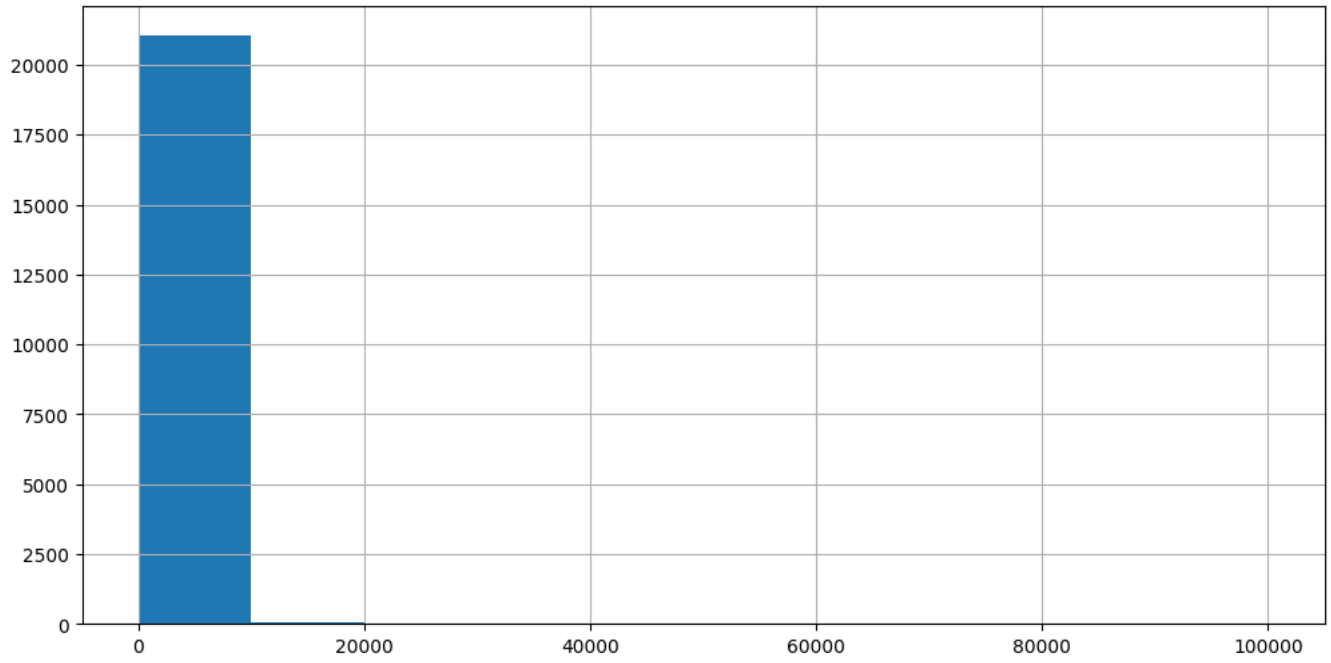
```
'霄云里8号是一家高端星级酒店公寓位于京城繁华的燕莎商圈附近汇聚着燕莎友谊商城等百货商场以及林立的高档写字楼这里临近北京国际展览中心农业展览馆是商务出差办事的便利下榻居所此外距离地铁10号线三元桥站也不远出租约起步价即到 酒店具有四大品质特征地处经济发达城市的中心区域品位高雅体现文化和艺术氛围配套设施满足白领精英人士商务休闲双重需求Ihotel网络时代的聪明酒店 这里属公寓式酒店格局性价比高酒店外的草坪绿化也增添了惬意之感闹中取静房间宽敞厨房设备均采用德国进口优质品牌提升整体居住舒适度同时房间内的10兆高速网络和大堂无线网络覆盖最适合现代人快节奏的网络生活'
```

## EDA (Explortary Data Analysis)

```
plt.figure(figsize=(12, 5))
df.boxplot(column=['prices'], vert=False)
plt.show()
```




```
plt.figure(figsize=(12, 6))
df['prices'].hist()
plt.show()
```




## Normalize prices and remove special symols

```
# Process price and remove special symbols and text, and normalize prices into range (0, 1.0)
df['prices'] = df.price.apply(lambda x:float(x.strip().replace(',', '').split('$')[-1]))
df['prices_norm'] = df.prices / df.prices.max()
# process description
df['new_description'] = df.description.apply(lambda x:remove_special_symbols(x, symbols))
```

```
# Drop duplicates on description and location the data needed
df.drop_duplicates(subset=['new_description'], inplace=True)

df_data = df.loc[:, ['new_description', 'prices', 'prices_norm']]
# df_data.to_csv('data/shanghai_processed_description_prices.csv', index=False)
# df_data.to_csv('data/beijing_processed_description_prices.csv', index=False)
```

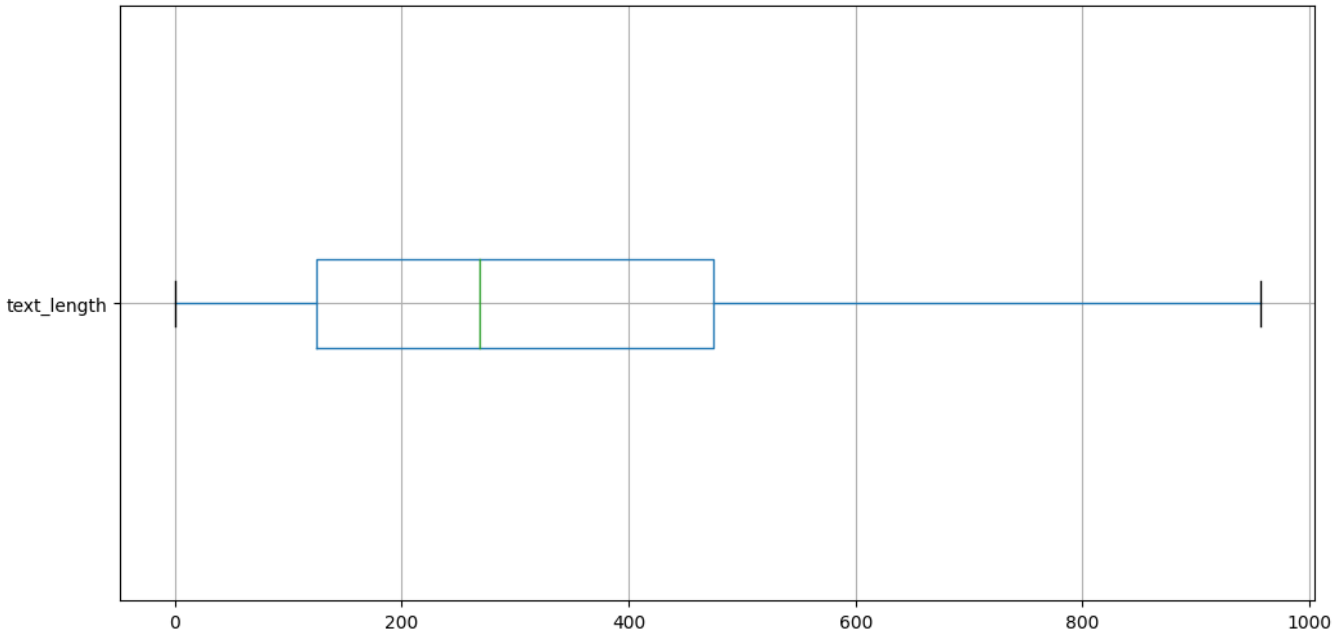
```
# Check max length of descriptions
df_data['text_length'] = df_data['new_description'].apply(len)
df_data.text_length.max()
```

```
957
```

## Check length of description after symbols removal

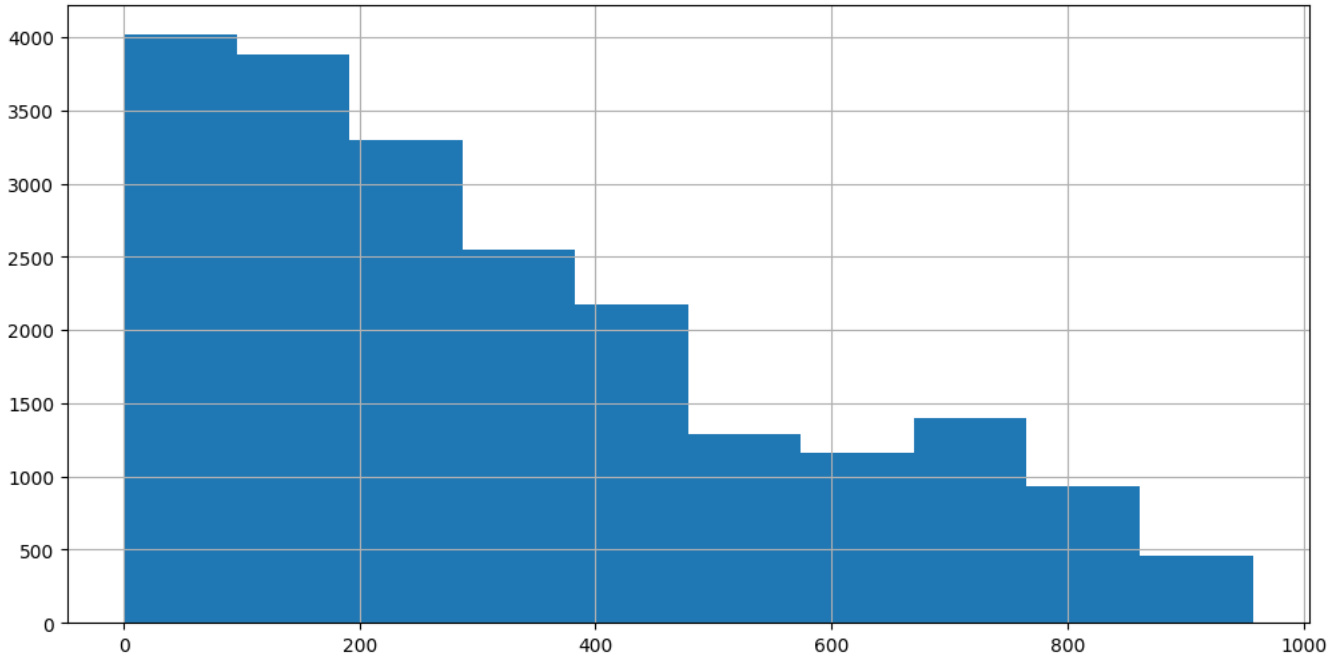
```
plt.figure(figsize=(12, 6))
df_data.boxplot(column=['text_length'], vert=False)
plt.show()
```

```
<Figure size 1200x600 with 0 Axes>
```




```
plt.figure(figsize=(12, 6))
df_data['text_length'].hist()
plt.show()
```

```
<Figure size 1200x600 with 0 Axes>
```




## Model preparation

```
BERT_PATH = 'pretrained/'
bert_tokenizer = BertTokenizer.from_pretrained(BERT_PATH)
bert_input = bert_tokenizer(
    df.new_description[0],
    padding='max_length',
    max_length=512,
    trunction=True,
    return_tensors='pt'
)
print(bert_input)
```

```
Keyword arguments {'trunction': True} not recognized.
```

```
{'input_ids': tensor([[   101,  82859,  10106,  10105,  27362,  12956,  10108,  62866,  20334,
          59791,  11490,  10117,  10751,  11846,  19342,  13354,  10940,  14155,
          11418,  10173,  26095,  21305,  23818,  17598, 105156,  15198,  87089,
          10117,  19555, 108076,  10301,  10188,  12492,  10147,  10729,  10114,
         101820,  10729,  10169,  32650,  28841,  13499,  14625,  10301,  10107,
          10111,  19361,  30113,  65218,  10117,  21905,  12077,  10380,  57797,
          35024,  18923,  10708,  19868,  30113,  10107,  10114,  13961,  10454,
          56673,  10948,  10105,    122,  36867,  10111,    123,  36867,  17861,
          13456,  13299,  13351, 103810,  10105,  64096,  88642,  10107,  27185,
          73696,  11898,  10105,  22921,  94982,  10111, 101328,  16405,  37025,
          10271,  59148,    169,  17684,  62893,  10567,  10473,  12647,  31324,
          54963,  10157,  11090,  11643,  16199,  10858,  46614,  10162,  27217,
          10112,  39639,  44357,  89096,  10169,  12364,  23467,  24302,  11337,
          13086,  13028,  28149,  19308,  11170,  88195,  11152,  10108,  10105,
          89096,  10107,  10301,  26989,  10169,  10244,  10147,  10729,  15336,
          14585,  14864,  10105,  12504,  10142,    170,  10457,  11703,  18708,
          11846,  24259,  88666,  10107,  20969, 104135,  10111,  13246,  12820,
          34279,  10160,  10105,  20756,  16836,  32362,  10171,  10301,  14579,
            102,      0,      0,      0,      0,      0,      0,      0,      0,
              0,      0,      0,      0,      0,      0,      0,      0,      0,
              0,      0,      0,      0,      0,      0,      0,      0,      0,
              0
```

```
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0]]), 'attention_mask': tensor([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1,
         1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
         1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
         1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
         1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
         1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
         1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0]])}
```

```
example_text = bert_tokenizer.decode(bert_input.input_ids[0])
example_text
```

```
'[CLS] Located in the core position of Chaoyang CBD The new highend community where businessmen congregate offers honor luxury
good lifestyle The room sizes are from 78m2 to 100m2 with spacious living ares and separate bedrooms The quality large sofas
easily turn into extra beds to freely switch between the 1br and 2br Standard modern Chinese style apartments the antique
furnitures completely throw off the oriental mysterious and charming Well it looks a bit retro but still quite trendyThe
spaceClosed fullyequipped kitchen with top brand facilities will make you amazing cooks some of the kitchens are connected with
30m2 terraces Its the best for bbq parties hightea gatherings couple romances and even just looking at the stars Opera glasses are
available [SEP] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD]
[PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD]
[PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD]
[PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD]
[PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD]
[PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD]
[PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD]
[PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD]
[PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD]
[PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD]
[PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD]
[PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD]
[PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD]
[PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD] [PAD]'
```

```
bert_model = BertModel.from_pretrained(BERT_PATH)
```

```
class BertPriceModel(nn.Module):
    def __init__(self, bert_pretrained_path, dropout=0.):
        super(BertPriceModel, self).__init__()
        self.bert = BertModel.from_pretrained(bert_pretrained_path)
        self.dropout = nn.Dropout(dropout)
        self.linear = nn.Linear(768, 1)
        self.sigmoid = nn.Sigmoid()

    def forward(self, input_id, mask):
        with torch.no_grad():
            _, pooled_output = self.bert(input_ids= input_id, attention_mask=mask,return_dict=False)
        dropout_output = self.dropout(pooled_output)
        linear_output = self.linear(dropout_output)
        final_layer = self.sigmoid(linear_output)
```

```
        return final_layer
```

## Prepare dataset

```
class PriceDataset(torch.utils.data.Dataset):
    def __init__(self, df):
        self.labels = [label for label in df['prices_norm']]
        self.texts = [bert_tokenizer(text,
                                padding='max_length',
                                max_length = 512,
                                truncation=True,
                                return_tensors="pt")
                      for text in df['new_description']]

    def classes(self):
        return self.labels

    def __len__(self):
        return len(self.labels)

    def get_batch_labels(self, idx):
        # Fetch a batch of labels
        return np.array(self.labels[idx])

    def get_batch_texts(self, idx):
        # Fetch a batch of inputs
        return self.texts[idx]

    def __getitem__(self, idx):
        batch_texts = self.get_batch_texts(idx)
        batch_y = self.get_batch_labels(idx)
        return batch_texts, batch_y
```

```
np.random.seed(42)
df_train, df_val, df_test = np.split(df_data.sample(frac=1, random_state=42),
                                     [int(.8*len(df_data)), int(.9*len(df_data))])

print(len(df_train),len(df_val), len(df_test))
```

```
16905 2113 2114
```

```
def RMSE(y, y_pred):
    with torch.no_grad():
        return torch.sqrt(((y - y_pred)**2).mean())
```

```
def train_one_epoch(epoch, model, dataloader, criterion, optimizer):
    rmse_meter = AverageMeter()
    loss_meter = AverageMeter()
    len_dataloader = len(dataloader)
    last_idx = len_dataloader - 1
    for i, (inputs, labels) in enumerate(dataloader):
        b = labels.size(0)
        labels = labels.float().cuda() # convert float64 to float32
        mask = inputs['attention_mask'].cuda()
        inputs_ids = inputs['input_ids'].squeeze(1).cuda()
        outputs = model(inputs_ids, mask)
        # print(outputs.dtype, labels.dtype)
        batch_loss = criterion(outputs.squeeze(1), labels)

        model.zero_grad()
        batch_loss.backward()
        optimizer.step()

        rmse = RMSE(labels, outputs)
        loss_meter.update(batch_loss.item(), b)
        rmse_meter.update(rmse.item(), b)
        if i > 0 and i % 50 == 0:
            msg =  f"Train: {epoch} [{i:>4d}/{len_dataloader} ({100. * i / last_idx:>3.0f}%)] "\
                   f"Loss: {loss_meter.val:.4g} ({loss_meter.avg:.3g}) "\
                   f"RMSE: {rmse_meter.val:>7.4f} ({rmse_meter.avg:>7.4f})"
            print(msg)
```

```python
train, val = PriceDataset(df_train), PriceDataset(df_val)
epoches = 5
batch_size = 32
# DataLoader根据batch_size获取数据，训练时选择打乱样本
train_dataloader = torch.utils.data.DataLoader(train, batch_size=batch_size, shuffle=True)
val_dataloader = torch.utils.data.DataLoader(val, batch_size=batch_size)

for epoch in range(epoches):
    train_one_epoch(epoch, model, train_dataloader, criterion, optimizer)
```

